

CS152 Assignment 2: The 8-puzzle

Before you turn in this assignment, make sure everything runs as expected. First, **restart the kernel** (in the menubar, select Kernel $\rightarrow$ Restart) and then run the test cells for each of the questions you have answered. Note that a grade of 3 for the A* implementation requires all tests in the "Basic Functionality" section to be passed. The test cells pass if they execute with no errors (i.e. all the assertions are passed).

Make sure you fill in any place that says `YOUR CODE HERE`. Be sure to remove the `raise NotImplementedError()` statements as you implement your code - these are simply there as a reminder if you forget to add code where it's needed.

Question 1

Define your `PuzzleNode` class below. Ensure that you include all attributes that you need to implement an A* search. If you wish, you can even include member functions, such as a function to generate successor states. Alternatively, you can code up this functionality later in the `solvePuzzle` function.

A* Search

The A* algorithm combines uniform-cost search and greedy best-first search by minimizing:

$f(n) = g(n) + h(n)$ where:

- $g(n)$ is the cost so far (depth of the node),
- $h(n)$ is the heuristic estimate of the cost to reach the goal

This ensures that if $h(n)$ is admissible (never overestimates the true remaining cost), A* will always return an optimal solution.

Problem Framing

- States: $n \times n$ boards (each with tiles $0 \dots n^2 - 1$)
- Initial state: the given board
- Actions: slide the blank up/down/left/right when possible
- Transition model: swapping 0 with an adjacent tile
- Step cost: 1 per move.
- Goal test: 0-first goal $[[0,1,2], [3,4,5], [6,7,8]]$ (generalized to $n \times n$).

This is a single-agent shortest-path search problem in a finite graph.

Node Structure and Puzzle Representation

Each state is represented as a nested list of lists, and the blank tile is represented by 0.

To manage search efficiently, I define a `PuzzleNode` class that encapsulates all the data needed for A* graph search, and each node corresponds to a unique puzzle configuration (state), storing:

- `board`: the current configuration of the puzzle
- `parent`: pointer used for path reconstruction
- `depth` $g(n)$: number of moves so far
- `cost` $f(n) = g(n) + h(n)$: total estimated cost, priority function used in A*
- `str()` - returns a human-readable grid representation,

The class also provides helper methods like:

- `find_position(value)`: returns the (row, column) coordinates of a tile
- `swap(pos1, pos2)`: returns a new board configuration with two tiles swapped
- `generate_children()`: generates all valid next states by sliding the blank tile in one of the possible directions

This structure keeps the code modular and allows A* to focus only on evaluating and expanding nodes.

Finally, because the overall algorithm implements graph search (using a closed set of visited states), each node represents a distinct state in the search graph.

```
In [ ]: # Import any packages you need here

import heapq # I use heapq to implement the A* frontier as a min-priority queue (always pop the lowest f = g + h)

#Now, define the class PuzzleNode:
class PuzzleNode:
    def __init__(self, board, parent=None, depth=0, cost=0):
        """
        One node in the A* search.
        - board: the current configuration as an n x n grid (0 is the blank)
        - parent: pointer to the node we came from (lets me reconstruct the solution path at the end)
        - depth: g(n) = number of moves from the start state to this node (path cost so far, each move costs 1)
        - cost: f(n) = g(n) + h(n). Stored here so the priority queue can compare nodes by total estimated cost

        A* expands nodes in order of lowest f. By storing g and f on the node, it's easy to push/pop from the heap.
        """
        self.board = board
        self.parent = parent
        self.depth = depth
        self.cost = cost #g + h, where h is given by the heuristic function

    def __lt__(self, other):
```

```

        """
        heapq requires a less than comparison. self.cost < other.cost ensures the heap pops the node with the smallest f(n)
        """
        return self.cost < other.cost

    def __str__(self):
        """
        Display the puzzle board in a human-readable grid format.
        For each row, I join the numbers with spaces, and then I join all rows with newline characters so the puzzle prints as a square
        """
        rows = []
        for row in self.board:
            line = " ".join(str(x) for x in row)
            rows.append(line)
        return "\n".join(rows)

    def find_position(self, value):
        """
        Find coordinates of a specific tile in this board.
        I use this to locate the blank (0) when generating successors.
        Time is O(n^2) which is fine for small puzzles used in the assignment.
        """
        for i in range(len(self.board)):
            for j in range(len(self.board)):
                if self.board[i][j] == value:
                    return (i, j)

    def swap(self, pos1, pos2):
        """
        Returns a new board with two tiles swapped (creates a new state instead of changing the
        current board, so other paths in the search aren't affected).
        """
        new_board = [row[:] for row in self.board]
        x1, y1 = pos1
        x2, y2 = pos2
        new_board[x1][y1], new_board[x2][y2] = new_board[x2][y2], new_board[x1][y1]
        return new_board

    def generate_children(self):
        """
        Generate all possible child states caused by sliding the blank tile up/down/left/right (depending on what's allowed)
        Later each child will get g(child) = g(current) + 1 (one slide); h(child) = heuristic(child.board);
        and f(child) which equals g(child) + h(child)
        """
        n = len(self.board)
        x, y = self.find_position(0) #locate the blank tile
        children = []

        # possible moves: up, down, left, right
        for dx, dy in [(-1,0), (1,0), (0,-1), (0,1)]:
            new_x, new_y = x + dx, y + dy
            if 0 <= new_x < n and 0 <= new_y < n: #ensuring it is within the bounds of the grid
                # create new board state by swapping blank with the neighbor
                new_board = self.swap((x, y), (new_x, new_y))
                children.append(new_board)

        return children

# Helper Functions
def flatten(board):
    """
    Turn a nested board into a flat list. I use this to store and compare states easily. Python can't put lists
    inside sets, so I flatten the board and turn it into a tuple when adding it to a closed set. It also helps
    checking validity and solvability, since those checks need to see all the tiles in one sequence.
    Since this is not "puzzle behavior", I decided to keep it as a helper function (not a class method).
    """
    return [x for row in board for x in row]

def make_goal_state(n):
    """
    Build the goal state with 0 at the top-left and increasing order.
    I generate the goal once (per n) and compare reached boards to it during A*.
    Since the goal is a property of the problem definition and not of a particular node
    it is kept outside the class.
    """
    goal = []
    count = 0
    for i in range(n):
        row = []
        for j in range(n):
            row.append(count)
            count += 1
        goal.append(row)
    return goal

```

Question 2

Define your heuristic functions using the templates below. Ensure that you extend the `heuristics` list to include all the heuristic functions you implement. Note that state will be given as a list of lists, so ensure your function accepts this format. You may use packages like `numpy` if you wish within the functions themselves.

Heuristic Functions

I implemented two admissible heuristics:

1. `h1` — Misplaced Tiles Counts how many tiles are not in their correct position. This heuristic is simple and always less than the true cost (since each misplaced tile needs at least one move to reach its goal). However, it doesn't reflect how far a tile is from its target.

2. h_2 — Manhattan Distance Sums the distances of each tile from its goal position (horizontal + vertical) Also admissible since each tile must move at least its Manhattan distance to reach the goal. It's a stronger heuristic because it provides a more accurate estimate of the remaining cost, so it usually expands fewer nodes. For this implementation I referred to (Pochmann, 2016) to realize that $goal_row = val // n$ and $goal_col = val \% n$ For instance, in a 3x3 board, tile 5 should be in row 1 ($5//3$) and column 2 ($5\%3$)

In practice, h_2 dominates h_1 since it is closer and more informative estimate, allowing A* to focus its search on more promising states, thereby expanding fewer nodes overall. However, a more detailed heuristic also has a higher computational cost per node, due to additional arithmetic and nested loops.

```
In [ ]: # Add any additional code here (e.g. for the memoization extension)

def h1(state):
    """
    This function returns the number of misplaced tiles, given the board state (ignoring the blank 0).
    Input:
        -state: the board state as a list of lists
    Output:
        -h: the number of misplaced tiles

    This heuristic never overestimates the number of moves required because each misplaced tile
    needs at least one move to reach its goal. Thus, h1 is admissible, meaning A* with h1 will
    always find an optimal solution.
    """
    n = len(state)
    flat_state = flatten(state)          # flatten the grid into one list
    flat_goal = [i for i in range(n * n)] # flattened version of the goal state for comparison
    h = 0

    for i in range(n * n):
        # Ignore the blank since it's not a real tile
        if flat_state[i] != 0 and flat_state[i] != flat_goal[i]:
            h += 1

    return h

def h2(board):
    """
    This function returns the Manhattan distance from the solved state, given the board state.
    Input:
        - state: the board state as a list of lists
    Output:
        - h: the Manhattan distance from the solved configuration

    For each tile (ignoring 0), computes how far it is from its goal position using by counting
    how many steps it must move horizontally and vertically with the formula:
    |current_row - goal_row| + |current_col - goal_col|.
    This heuristic is admissible since it never overestimates the true number of moves:
    each tile must at least move as many times as its distance from the goal.
    """
    n = len(board)
    total = 0
    for i in range(n):
        for j in range(n):
            val = board[i][j]
            if val != 0: #skip blank tile
                goal_row = val // n # gives the goal row
                goal_col = val % n # gives the goal col Reference: (Pochmann, 2016) for idea to use mod
                total += abs(i - goal_row) + abs(j - goal_col) # horizontal + vertical steps

    return total

# Extra heuristic for the extension. If implemented, modify the function below
def h3(state):
    """
    This function returns a heuristic that dominates the Manhattan distance, given the board state
    Input:
        -state: the board state as a list of lists
    Output:
        -h: the Heuristic distance of the state from its solved configuration
    """
    return 0

# If you implement more than 3 heuristics, then add any extra heuristic functions onto the end of this list.
heuristics = [h1, h2, h3]
```

Question 3

Code up your A* search using the SolvePuzzle function within the template below. Please do not modify the function header, otherwise the automated testing will fail. You may define other functions or import packages as needed in this cell or by adding additional cells.

Design Choices and Justification

- The frontier (open_list) is implemented as a min-heap (priority queue) using heapq, ordered by the node's cost =g+h. This makes A* always expand the node with the lowest f(n).
- The closed set stores explored states (stored as tuples) to avoid revisiting them. This prevents re-expansions and cycles, and each unique state in the state space is expanded at most once (graph search).
- Each iteration expands the node with the lowest f(n) and generates its children.
- The search terminates when the current node equals the goal state.

To track performance:

- exp: counts the total number of nodes expanded.
- max_frontier: records the maximum size of the frontier (measures space complexity)

Solvability Check

Borrowing the logic from (Princeton University, n.d.), see References:

In order to check if all boards are solvable, we count "inversions."

- For odd-sized boards (n odd): solvable if the inversion count is even
- For even-sized boards (n even): solvable if (inversions + blank_row) is odd.

This rejects unreachable instances early, saving search.

```
In [ ]: # Import any packages or define any helper functions you need here

def valid_input(board):
    """
    Helper function that checks that the board contains all values from 0 to n^2-1
    exactly once, ensuring the puzzle is valid before starting A* search.
    If there are missing or duplicate numbers then the puzzle doesn't exist in
    the defined problem space and cannot be solved.

    Returns True if the board is valid, otherwise False.
    """
    flat = flatten(board) #flatten for easy comparison
    n = len(board)
    return sorted(flat) == list(range(n*n))

def is_solvable(board):
    """
    Checks whether the given puzzle is solvable with the 0 first goal state.
    The solvability test (Princeton, n.d.) is based on counting "inversions."
    These are pairs of tiles that are in the wrong order relative to each other.
    So for pair (a,b), a appears before b but a>b.

    For odd-sized boards (nxn board where n is an odd number):
    - the puzzle is solvable if the number of inversions is even.

    For even-sized boards:
    - the puzzle is solvable if the number of inversions + row of the blank tile
    from the bottom is odd.

    Returns True if solvable, otherwise False.
    """
    n = len(board)
    flat = flatten(board)
    inversions = 0

    # Count inversions
    for i in range(len(flat)):
        for j in range(i + 1, len(flat)):
            if flat[i] == 0 or flat[j] == 0: # skip blank tile
                continue
            if flat[i] > flat[j]: # i comes before j, yet i > j
                inversions += 1

    if n % 2 == 1:
        # Odd-size puzzles are solvable if inversion count is even
        return inversions % 2 == 0
    else:
        # For even-size puzzles need to check blank position from bottom
        blank_index = flat.index(0) #find index
        blank_row_from_top = blank_index // n #find row
        blank_row_from_bottom = n - 1 - blank_row_from_top # based from bottom since 0-first goal state

        # Solvable if inversions + blank_row_from_bottom is odd
        return (inversions + blank_row_from_bottom) % 2 == 1

# -----
# A* Search Algorithm

def solvePuzzle(state, heuristic):
    """
    Solves the n^2-1 puzzle for any n > 2 (although it may take too long for n > 4)).
    A* is an informed search algorithm that combines the actual path cost g
    and an estimated cost to the goal h to decide which node to expand next
    (the node with the smallest f = g + h value, the top of min heap).
    The algorithm guarantees an optimal solution if the heuristic is admissible.

    Inputs:
    -state: The initial state of the puzzle as a list of lists
    -heuristic: a function that estimates the cost from given to goal state (one from question 2).
    Outputs:
    -steps: The number of steps to optimally solve the puzzle (excluding the initial state)
    -exp: The number of nodes expanded during the search to reach the solution.
        In other words, how much of the state space was explored.
    -max_frontier: The maximum number of nodes ever stored in the priority queue (useful to estimate memory complexity).
    -opt_path: The optimal path as a list of list of lists. That is, opt_path[:,i] should give a list of lists
        that represents the state of the board at the ith step of the solution.
    -err: An error code. If state is not of the appropriate size and dimension, return -1.
        If the state is not solvable, then return -2
        Success, return 0.
    """
    n = len(state)

    # Validate input
    if not valid_input(state): # correct tiles
        return 0, 0, 0, None, -1
    if not is_solvable(state): # is solvable
```

```

    return 0, 0, 0, None, -2

goal_state = make_goal_state(n) #target configuration using helper function
open_list = [] # priority queue (frontier)
closed = set() # explored states; using a set allows fast O(1) lookups to avoid re-expanding nodes

# Calculate heuristic cost which equals f initially
start_h = heuristic(state)
start_state = PuzzleNode(state, None, 0, start_h) #depth=0, cost=f=h
heapq.heappush(open_list, start_state) # add start node to frontier; heap ensures nodes are ordered by lowest f

exp = 0 # number of nodes expanded
max_frontier = 1 # tracking frontier size

# Main Loop
while open_list: #while frontier is not empty
    current = heapq.heappop(open_list) #pop the node with smallest f
    exp += 1

    # Check if reached goal
    if current.board == goal_state:
        # Reconstruct the solution path by following parent links
        path = []
        node = current # refers to goal node
        while node:
            path.append(node.board)
            node = node.parent
        path.reverse() # since the list is backwards (goal-->start), this flips it
        steps = len(path) - 1 # number of moves, excluding the start state
        return steps, exp, max_frontier, path, 0 # success

    # Mark this configuration as visited
    closed.add(tuple(flatten(current.board))) #turn to tuple so can be added to the set

    # Generate child states
    for new_board in current.generate_children():
        if tuple(flatten(new_board)) in closed: #skip states that were already explored
            continue #which is what makes this a graph search rather than a tree search

        g = current.depth + 1 # cost to reach the new node
        h = heuristic(new_board) # heuristic estimate from the new state to the goal
        new_node = PuzzleNode(new_board, current, g, g + h)
        heapq.heappush(open_list, new_node) #push new node into frontier

    # Update maximum frontier size
    if len(open_list) > max_frontier:
        max_frontier = len(open_list)

# No solution found
return 0, exp, max_frontier, None, -2

```

Extension Questions

The extensions can be implemented by modifying the code from Q2-3 above appropriately.

1. **Initial state solvability:** Modify your SolvePuzzle function code in Q3 to return -2 if an initial state is not solvable to the goal state.
2. **Extra heuristic function:** Add another heuristic function (e.g. pattern database) that dominates the misplaced tiles and Manhattan distance heuristics to your Q2 code.
3. **Memoization:** Modify your heuristic function definitions in Q2 by using a Python decorator to speed up heuristic function evaluation

There are test cells provided for extension questions 1 and 2.

Basic Functionality Tests

The cells below contain tests to verify that your code is working properly to be classified as basically functional. Please note that a grade of **3** on #aicoding and #search as applicable for each test requires the test to be successfully passed. **If you want to demonstrate some other aspect of your code, then feel free to add additional cells with test code and document what they do.**

In []: **## Test for state not correctly defined**

```

incorrect_state = [[0,1,2],[2,3,4],[5,6,7]]
_,_,_,err = solvePuzzle(incorrect_state, lambda state: 0)
assert(err == -1)

```

In []: **## Heuristic function tests for misplaced tiles and manhattan distance**

```

# Define the working initial states
working_initial_states_8_puzzle = ([[2,3,7],[1,8,0],[6,5,4]], [[7,0,8],[4,6,1],[5,3,2]], [[5,7,6],[2,4,3],[8,1,0]])

# Test the values returned by the heuristic functions
h_mt_vals = [7,8,7]
h_man_vals = [15,17,18]

for i in range(0,3):
    h_mt = heuristics[0](working_initial_states_8_puzzle[i])
    h_man = heuristics[1](working_initial_states_8_puzzle[i])
    assert(h_mt == h_mt_vals[i])
    assert(h_man == h_man_vals[i])

```

In []: **# Visualize that h2 dominates**

```

test_state = [[2,3,7],[1,8,0],[6,5,4]]

```

```

steps_h1, exp_h1, frontier_h1, _, _ = solvePuzzle(test_state, h1)
steps_h2, exp_h2, frontier_h2, _, _ = solvePuzzle(test_state, h2)

print(f"h1 (Misplaced Tiles): Expanded {exp_h1} nodes, frontier max size = {frontier_h1}, steps = {steps_h1}")
print(f"h2 (Manhattan): Expanded {exp_h2} nodes, frontier max size = {frontier_h2}, steps = {steps_h2}")

```

```

In [ ]: ## A* Tests for 3 x 3 boards
## This test runs A* with both heuristics and ensures that the same optimal number of steps are found
## with each heuristic.

# Optimal path to the solution for the first 3 x 3 state
opt_path_soln = [[2, 3, 7], [1, 8, 0], [6, 5, 4]], [[2, 3, 7], [1, 8, 4], [6, 5, 0]],
                [[2, 3, 7], [1, 8, 4], [6, 0, 5]], [[2, 3, 7], [1, 0, 4], [6, 8, 5]],
                [[2, 0, 7], [1, 3, 4], [6, 8, 5]], [[0, 2, 7], [1, 3, 4], [6, 8, 5]],
                [[1, 2, 7], [0, 3, 4], [6, 8, 5]], [[1, 2, 7], [3, 0, 4], [6, 8, 5]],
                [[1, 2, 7], [3, 4, 0], [6, 8, 5]], [[1, 2, 0], [3, 4, 7], [6, 8, 5]],
                [[1, 0, 2], [3, 4, 7], [6, 8, 5]], [[1, 4, 2], [3, 0, 7], [6, 8, 5]],
                [[1, 4, 2], [3, 7, 0], [6, 8, 5]], [[1, 4, 2], [3, 7, 5], [6, 8, 0]],
                [[1, 4, 2], [3, 7, 5], [6, 0, 8]], [[1, 4, 2], [3, 0, 5], [6, 7, 8]],
                [[1, 0, 2], [3, 4, 5], [6, 7, 8]], [[0, 1, 2], [3, 4, 5], [6, 7, 8]]

astar_steps = [17, 25, 28]
for i in range(0,3):
    steps_mt, expansions_mt, _, opt_path_mt, _ = solvePuzzle(working_initial_states_8_puzzle[i], heuristics[0])
    steps_man, expansions_man, _, opt_path_man, _ = solvePuzzle(working_initial_states_8_puzzle[i], heuristics[1])
    # Test whether the number of optimal steps is correct and the same
    assert(steps_mt == steps_man == astar_steps[i])
    # Test whether or not the manhattan distance dominates the misplaced tiles heuristic in every case
    assert(expansions_man < expansions_mt)
    # For the first state, test that the optimal path is the same
    if i == 0:
        assert(opt_path_mt == opt_path_soln)

```

```

In [ ]: ## A* Test for 4 x 4 board
## This test runs A* with both heuristics and ensures that the same optimal number of steps are found
## with each heuristic.

working_initial_state_15_puzzle = [[1,2,6,3],[0,9,5,7],[4,13,10,11],[8,12,14,15]]
steps_mt, expansions_mt, _, _, _ = solvePuzzle(working_initial_state_15_puzzle, heuristics[0])
steps_man, expansions_man, _, _, _ = solvePuzzle(working_initial_state_15_puzzle, heuristics[1])
# Test whether the number of optimal steps is correct and the same
assert(steps_mt == steps_man == 9)
# Test whether or not the manhattan distance dominates the misplaced tiles heuristic in every case
assert(expansions_mt >= expansions_man)

```

Extension Tests

The cells below can be used to test the extension questions. Memoization if implemented will be tested on the final submission - you can test it yourself by testing the execution time of the heuristic functions with and without it.

```

In [ ]: ## Puzzle solvability test

unsolvable_initial_state = [[7,5,6],[2,4,3],[8,1,0]]
_,_,_,_,err = solvePuzzle(unsolvable_initial_state, lambda state: 0)
assert(err == -2)

```

```

In [ ]: ## Extra 4x4 solvability test

solvable_4x4 = [
    [1, 2, 0, 3],
    [4, 5, 6, 7],
    [8, 9, 10, 11],
    [12, 13, 14, 15]
]
steps, exp, frontier, path, err = solvePuzzle(solvable_4x4, h1)
assert err == 0

unsolvable_4x4 = [
    [0, 2, 1, 3],
    [4, 5, 6, 7],
    [8, 9, 10, 11],
    [12, 13, 14, 15]
]
_, _, _, _, err = solvePuzzle(unsolvable_4x4, lambda state: 0)
assert err == -2

```

```

In [ ]: ## Extra heuristic function test.
## This tests that for all initial conditions, the new heuristic dominates over the manhattan distance.

dom = 0
for i in range(0,3):
    steps_new, expansions_new, _, _, _ = solvePuzzle(working_initial_states_8_puzzle[i], heuristics[2])
    steps_man, expansions_man, _, _, _ = solvePuzzle(working_initial_states_8_puzzle[i], heuristics[1])
    # Test whether the number of optimal steps is correct and the same
    assert(steps_new == steps_man == astar_steps[i])
    # Test whether or not the manhattan distance is dominated by the new heuristic in every case, by checking
    # the number of nodes expanded
    dom = expansions_man - expansions_new
    assert(dom > 0)

```

```

In [ ]: ## Memoization test - will be carried out after submission

```

References

8-Puzzle. Princeton University. (n.d.). <https://www.cs.princeton.edu/courses/archive/fall17/cos226/assignments/8puzzle/index.html>

GeeksforGeeks. (2025, July 23). 8 puzzle problem in Ai. <https://www.geeksforgeeks.org/artificial-intelligence/8-puzzle-problem-in-ai/>

GenericUser01, & Pochmann, S. (2016, September 29). Calculating the Manhattan distance in the eight puzzle. Stack Overflow. <https://stackoverflow.com/questions/39759721/calculating-the-manhattan-distance-in-the-eight-puzzle>